

Understanding Model Context Protocol (MCP) and Building Custom MCP Tools

What is the Model Context Protocol (MCP)?

The **Model Context Protocol (MCP)** is an open standard (introduced by Anthropic in late 2024) that defines a universal way for AI models to connect with external tools, data sources, and context ¹ ². Think of MCP as a **“USB-C port” for AI applications** – it provides a standardized interface through which any AI assistant (LLM) can plug into various functionalities without custom integration for each tool ³. In practical terms, MCP uses a client-server architecture and JSON-RPC 2.0 messages (inspired by how the Language Server Protocol works) to let AI systems read files, call functions, or inject prompts via external modules called *MCP servers* ⁴. This approach replaces the siloed, vendor-specific connectors (like older function-calling APIs or one-off plugins) with a **model-agnostic, interoperable protocol** that multiple AI providers support ⁵ ¹. Major AI platforms including OpenAI, Google DeepMind, and Microsoft have adopted MCP, which means tools built with MCP can be used across different AI agents and environments ⁶.

Key capabilities of MCP: An MCP server can expose three types of assets to an AI model ⁷:

- **Resources:** file-like data or documents that the AI can read (e.g. the contents of a file or a database query result).
- **Tools:** functions or operations that the AI can call to perform actions (e.g. execute code, query an API, run a shell command). Tools may take arguments and return results to the model.
- **Prompts:** pre-defined prompt templates that can guide or structure the AI’s behavior in certain tasks.

Most often, discussions around MCP focus on **tools**, since they let LLMs actively perform tasks. MCP essentially provides a *universal tool API* for AI agents. Instead of each AI having hard-coded plugins, any MCP-compliant agent can discover and use any MCP-compliant tool server. This standardization greatly improves interoperability and scalability – for example, an AI agent could seamlessly access a filesystem, call web APIs, query databases, or use cloud services, as long as there are MCP servers for those functions ⁸ ⁹. In fact, numerous pre-built MCP connectors exist (for Google Drive, Slack, GitHub, databases, etc.), and companies like Block, Zed, Replit, and Sourcegraph have integrated MCP to give AI assistants real-time access to coding environments and other data ¹⁰ ¹¹.

OpenAI has embraced MCP in its own tooling. For instance, the OpenAI Agents SDK (released in 2025) can incorporate MCP servers as tools for an agent ¹² ¹³. Likewise, OpenAI’s Responses API (for building chain-of-thought agents) added support for remote MCP servers, allowing GPT-4 models to connect to any MCP tool with just a few lines of code ¹⁴ ¹⁵. In short, MCP has quickly become a **common bridge** between LLMs and the outside world, analogous to how OpenAPI (Swagger) standardized RESTful APIs ¹⁶.

How MCP Works: Client–Server Tool Architecture

MCP follows a **client–server model** in which AI agents (clients) communicate with external tool providers (servers) in a structured way ¹⁷. The *host process* (e.g. an AI assistant environment like Claude Desktop, ChatGPT with an agent, or an IDE plugin) can connect to one or more MCP servers. Each MCP server is a program (which could run locally as a subprocess or remotely over HTTP) that implements the MCP interface and offers a set of tools/resources. When the host connects to a server, it spawns an MCP **client** handler for that connection, which manages the JSON-RPC communication and isolates the server's execution environment ¹⁸.

Tool discovery and invocation: Once connected, the agent will query the server for its available tools (usually by calling a `list_tools` method under the hood). The server responds with a standardized description of each tool – typically the tool's name, description, input parameters, and output format (schema). This lets the AI model “know” what functions it can call, similar to how function calling in OpenAI works but in a model-agnostic format. At runtime, the workflow looks like this:

1. **LLM decides to use a tool:** The AI (during its chain-of-thought reasoning) analyzes the user's request and the available MCP tools. It may decide that calling a tool is necessary (e.g. to fetch information or perform an action).
2. **Agent calls the tool via MCP:** The agent (MCP client) invokes the chosen tool by sending a JSON-RPC request (`call_tool`) to the MCP server, with the function name and arguments.
3. **Server executes the function:** The MCP server receives the request and runs the corresponding tool's logic. This could involve reading files, calling an external API, running code, etc., depending on the tool. The server then returns the function's result (or error) in a JSON-RPC response.
4. **LLM uses the result:** The agent passes the tool's output back into the LLM's context. The model incorporates that data into its answer to the user. For example, the model might say “The tests failed with 2 errors...” after getting a result from a `run_tests` tool.
5. **Conversation continues:** The AI can decide to call more tools if needed, or finalize its answer. From the user's perspective, the agent has seamlessly fetched external info or performed actions and then responded in natural language.

This flow is supported automatically by frameworks like OpenAI's Agents SDK: when you attach an MCP server to an agent, the SDK will handle listing the server's tools and routing tool calls to it as the LLM requests ¹⁹. In other words, *the MCP client makes the tools “visible” to the AI and mediates the execution*. The security model usually requires **user approval** for certain tool uses (especially if they can modify data), and the host can enforce permission or sandboxing for safety ²⁰. The communication itself is stateless JSON-RPC over either a **STDIO stream** (for local tools launched as subprocesses) or **HTTP/SSE** (for remote tool servers) as defined by the MCP spec ²¹ ²².

Building Custom MCP Tools (MCP Servers) from Scratch

Building an MCP tool means implementing an **MCP server** that exposes one or more custom functions for the AI to use. The MCP specification is open, and there are official SDKs in multiple languages to simplify

development ²³. For example, Anthropic provides reference SDKs in Python, Node.js, Java/Kotlin, and C# for building MCP servers ²⁴. Here's a general process to create your own MCP tools:

1. **Plan the tool's functionality and interface:** Decide what action or data your tool will provide, and what its API should look like (name, input parameters, output). For instance, you might want a tool called `"build_project"` that takes a project path and runs a build, or `"run_tests"` that executes a test suite and returns a summary. Keep in mind the tool should be described clearly to the LLM, and inputs/outputs should be serializable (JSON-friendly).
2. **Set up an MCP server project:** Using your language of choice, initialize a new project and add the MCP server library. For example, in Python you would install the `mcp` package (e.g. via pip) and possibly a CLI helper like `uv` to run the server ²⁵ ²⁶. In Node, you might install `@modelcontextprotocol/...` packages. The SDK will handle the low-level protocol (JSON-RPC messaging, tool registration) so you can focus on writing the tool logic.
3. **Define your tool functions:** Write the functions that you want to expose. With the SDK, this often involves a decorator or registration call. For example, using the Python SDK's FastMCP, you can do:

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("my_tools") # create an MCP server instance with a name

@mcp.tool()
def build_project(path: str) -> str:
    """Build the Xcode project at the given path and return a summary of
    results.

    Args:
        path: Filesystem path to the Xcode project directory.
    """
    # ... (invoke build, capture output) ...
    return summary_text
```

In this example, the `@mcp.tool()` decorator automatically registers the function as an MCP tool. The function's signature and docstring are used to generate the tool schema (name, parameters, and description) that will be shown to the LLM ²⁷ ²⁸. The implementation can call external processes or libraries – for instance, you might use Python's `subprocess` to run `xcodebuild` and then parse the output for errors or test results. The return value (here a string `summary_text`) is what the AI will receive. **Note:** For STDIO-based servers, avoid printing to stdout inside your tool functions (use logging to stderr if needed), since stdout is reserved for the MCP JSON-RPC channel ²⁹.

1. **Run the MCP server:** Finally, start the server so it listens for incoming requests. With most SDKs this is a one-liner. In our Python example, you would call something like `mcp.run(transport="stdio")` if launching locally ³⁰ (or `mcp.run_http(port=...)` for an HTTP server). This turns your script into a live MCP server that can accept tool calls. If you execute the program (e.g. `python build_tool.py` or via the `uv` CLI as shown in the MCP docs), it will wait for an AI client to connect.

2. **Integrate with an AI agent:** To use your new MCP tool, you need to connect it to an AI that supports MCP. In practice, this could mean configuring the host application or agent to launch your server. For example, with OpenAI's Agents SDK, you can add a local MCP server via STDIO by specifying the command to run your server program ³¹. Once configured, the agent will automatically list the tools from your server at runtime and make them available to the model ¹⁹. In a custom setup, you might manually run the MCP server and then provide its address (or STDIO pipe) to the AI client. OpenAI's Responses API also allows developers to specify MCP servers by URL; for instance, you can tell the API about your tool with a snippet like:

```
response = client.responses.create(  
    model="gpt-4.1",  
    tools=[  
        {"type": "mcp",  
         "server_label": "build_tools",  
         "server_url": "http://localhost:5000" # if you ran an HTTP server  
        },  
    ],  
    input="Build and test the project..."  
)
```

This would let the GPT-4 model know that a toolset called “build_tools” is available at the given endpoint ¹⁵. The model could then choose to call functions from that MCP server as needed to fulfill the task. (If using a local STDIO server instead, the configuration is usually handled by the agent framework rather than a URL.)

By following these steps, you essentially create a **modular plugin** for AI agents. Your MCP server can contain one or many related tools. For example, you might include both `build_project` and `run_tests` tools in the same server if they are related (so the AI can use them together). The MCP spec allows rich tool descriptions, so be sure to document what each tool does and what it returns – the AI will read those descriptions to decide when to use the tool.

Example Use-Case: Build and Test Automation via MCP

To illustrate, imagine we want our AI coding assistant (e.g. OpenAI Codex or GPT-4 in agent mode) to compile an Xcode project and run its unit tests on command. We can implement this as a custom MCP server:

- **Tool:** `build_and_test(project_path: str) -> str` – This tool could call Apple's Xcode command-line tool (`xcodebuild`) to build the project at the given path and run its test suite. The function would capture the output (which might be very verbose) and distill it into a **summary of results**. For instance, it might report how many tests passed/failed, and list any compilation errors or test failures (excluding trivial warnings). The summary string is returned to the AI. This way, the LLM doesn't get overwhelmed with log details, but it has the key information to inform the user ³². If needed, the full raw log could be saved as a resource (e.g. the tool could write to a file that the AI can access via a `read_file` resource tool, for more detailed inspection on demand).

- **Additional tools (optional):** We could also provide specialized tools for “different views” of the build output. For example, a `get_last_build_errors()` tool might return just the error messages from the most recent build, or `list_tests()` might return a list of all unit tests and their status. These would be separate functions in the MCP server, allowing the AI to query specific information. In practice, one might simply implement `build_and_test` to return structured data (e.g. JSON with counts and error lists) so the AI can format it, but exposing multiple narrower tools is also possible. The MCP framework is flexible – you can design the interface that makes the most sense for your workflow.

Once our **“Build Tools” MCP server** is running, we integrate it with the AI agent. Now, when a user asks “Can you build the project and run tests?”, the AI will recognize that it has an available tool to do this. It will call `build_and_test`, the MCP server will execute the Xcode build in the background, and the summarized results will be returned for the AI to present. Thanks to MCP’s standardized interface, all of this occurs in a safe, controlled manner: the AI didn’t need direct shell access or special privileges beyond the tool’s scope, and any other AI agent that understands MCP could also use this tool in the future ^{9 33}.

Conclusion

In summary, MCP provides a powerful **abstraction layer for AI-tool integrations**. It allows developers to **build custom tools** (MCP servers) in a variety of languages and expose them to AI models in a consistent way. OpenAI’s Codex (and the broader Agents/Responses ecosystem) can leverage MCP tools easily, as OpenAI has native support for the protocol ¹⁵. By adding an “MCP” folder to your project with a custom server, you enable your AI agent to perform new actions like building code, running tests, querying databases, or anything else you implement. The steps involve writing the tool’s code, wrapping it with the MCP interface (using an SDK), and configuring your AI agent to connect to it. Once set up, the AI will dynamically discover the tool and use it to fulfill user requests whenever appropriate ¹⁹. This modular approach means you can continually extend your AI’s capabilities by adding or updating MCP tools, all while relying on a robust standard that ensures compatibility across different models and platforms.

Sources:

- Anthropic, *Introducing the Model Context Protocol (MCP)* – Anthropic News (Nov 2024) ^{2 34}
- Wikipedia, “Model Context Protocol” – background and adoption of MCP ^{1 4}
- OpenAI, *OpenAI Agents SDK Documentation: MCP Integration* – on using MCP servers in OpenAI’s agent framework ^{3 35}
- OpenAI, *New tools and features in the Responses API* (May 2025) – announcement of MCP support in OpenAI’s API ^{14 15}
- *Model Context Protocol – Quickstart Guide* (modelcontextprotocol.io) – tutorial on building a custom MCP server (e.g. a weather tool) ^{7 36}
- Docker Blog, *Simplifying AI Apps with MCP* – overview of MCP use-cases and tooling examples ^{8 9}
- Anthropic Community post, *MCP Deep Dive* – explanation of MCP vs. function calling and architecture ^{37 18}
- Composio Blog, *Building MCP Agents with OpenAI SDK* – example of integrating third-party MCP tool servers ^{12 38}
- OpenAI Community forum – discussions on MCP as an open standard for agent tools ³⁹ (general conceptual alignment)

1 4 6 11 16 21 33 **Model Context Protocol - Wikipedia**

https://en.wikipedia.org/wiki/Model_Context_Protocol

2 10 32 34 39 **Introducing the Model Context Protocol \ Anthropic**

<https://www.anthropic.com/news/model-context-protocol>

3 13 19 22 31 35 **Model context protocol (MCP) - OpenAI Agents SDK**

<https://openai.github.io/openai-agents-python/mcp/>

5 17 18 20 37 **Anthropic's Model Context Protocol (MCP): A Deep Dive for Developers | by Amanatullah | Medium**

<https://medium.com/@amanatulla1606/anthropics-model-context-protocol-mcp-a-deep-dive-for-developers-1d3db39c9fdc>

7 24 25 26 27 28 29 30 36 **Build an MCP Server - Model Context Protocol**

<https://modelcontextprotocol.io/quickstart/server>

8 9 **The Model Context Protocol: Simplifying Building AI apps with Anthropic Claude Desktop and Docker | Docker**

<https://www.docker.com/blog/the-model-context-protocol-simplifying-building-ai-apps-with-anthropic-claude-desktop-and-docker/>

12 38 **Building MCP agents with OpenAI Agents SDK - Composio**

<https://composio.dev/blog/building-mcp-agents-with-openai-agents-sdk>

14 15 **New tools and features in the Responses API | OpenAI**

<https://openai.com/index/new-tools-and-features-in-the-responses-api/>

23 **Introduction - Model Context Protocol**

<https://modelcontextprotocol.io/docs/getting-started/intro>